

COMPILER DESIGN

UNIT-3

[CS - 603]

TYPE CHECKING & RUN TIME ENVIRONMENT

SYLLABUS -

Type Checking: type system, specification of simple type checker, equivalence of expression, types, type conversion, overloading of functions & operations, polymorphic function.

Runtime Environment: storage organization, storage allocation strategies, parameter passing, dynamic storage allocation, symbol table, error detection & recovery, Ad-hoc and systematic methods.

TYPE CHECKING

TYPE SYSTEM -

A Type System is a set of rules used by a compiler to assign & check data types of variables, expressions & functions in a program.

It ensures that operations are performed on compatible data types.

Purpose of Type System-

- Detect errors at compile time.
- Ensure type safety
- Prevent invalid operations
- Improve program reliability
- Help in memory management.

It helps to detect errors during compilation or execution.

SPECIFICATION OF SIMPLE TYPE CHECKER [June 2025]

A Simple Type checker is a component of the compiler that verifies whether the types of variables & expressions are used correctly according to the language rules.

It works during the semantic analysis phase.

Purpose -

- Detect Type errors.
- Ensure valid operations
- Enforce type rules of the language.
- Improve program correctness.

Basic Idea:

The Type Checker -

- Assigns a type to each variable & expression.
- Checks whether operations are type-compatible.
- Reports error if types are invalid.

Specification -

A simple type checker is specified using type rules (also called semantic rules).

1. Type Assignment Rule → Each variable must have a declared type.

Ex:-

int a;

int b;

float c;

2. Expression Type Rule → The type of an expression depends on operands.

Ex:-

int + int → int

int + float → float.

3. Operator Compatibility Rule → Operators can only be applied to valid operand types.

Ex:-

int + int [Valid]

int + char (converted) [Valid]

int + string [Invalid]

4. Assignment Rule → left-hand side & right-hand side types must be compatible.

Ex:-

int a;

a = 10;

✓

a = 5.5;

✓ (converted)

a = hi;

✗ error

5. Function Call Rule → Number & type of arguments must match parameters.

Ex:-

int add(int a, int b);

add(5, 10);

✓

add(5, 2.5);

✗ (type mismatch).

Error Detection

Type checker detects:

- Type mismatch
- Invalid operations
- wrong function arguments.

Advantages

- Early error detection
- Improves program safety
- Helps compiler optimization

EQUIVALENCE OF EXPRESSION

Equivalence of expression (or type equivalence) means determining whether two expressions or types are considered the same by the compiler.

It is used in type checking to verify whether operations or assignments are valid.

Types of Equivalence -

1. Name Equivalence
2. Structural Equivalence

1) Name Equivalence $\rightarrow$ Two types are equivalent only if they have the same name.

Ex:-

```
typedef int A;
```

```
typedef int B;
```

```
A x;
```

```
B y;
```

$x = y;$ // Not equivalent (different names)

2) Structural Equivalence $\rightarrow$ Two types are equivalent if their structure is the same, regardless of name.

Ex:-

```
struct A { int x; };
```

```
struct B { int x; };
```

```
A a;
```

```
B b;
```

$a = b$ // $\checkmark$ Equivalent (same structure)

Used In-

- Assignment statements
- Function arguments
- Expression evaluation
- Operator usage.

Types of Data Types -

Basic Types → int, float, char

Derived Types → array, pointer

User-defined → struct, union

TYPE CONVERSION

Type Conversion is the process of converting a value from one data type to another. So that operations between different types can be performed correctly.

It is used by the compiler to ensure type compatibility in expressions & assignments.

Need for Type Conversion -

- To perform operations between different data types.
- To avoid type mismatch errors.
- To maintain precision & correctness.
- To make programs flexible.

TYPES OF TYPE CONVERSION

1) Implicit Type Conversion (Automatic Conversion)

Done automatically by the compiler.

No programmer intervention required.

Also called coercion.

Example:-

```
int a = 10;
```

```
float b = 5.5;
```

```
float c = a + b; // int to float automatically.
```

Here, a is converted to float before addition.

Advantages-

- Easy to use.
- Reduces coding effort.

Disadvantages-

- may cause loss of precision
- Sometimes leads to unexpected result.

2. Explicit Type Conversion (Type Casting)

- Done manually by the programmer.
- Also called Type casting.

Syntax:-

(type) expression

Example:-

```
int a = 5;
```

```
float b;
```

```
b = (float) a; // explicit conversion.
```

Advantages

- Full Control over conversion
- Reduces unexpected errors.

Disadvantages

- Require programmer effort
- Wrong casting may cause errors.

Example:-

```
int a = 5;
```

```
float b = 4.5;
```

```
float c = a + b; // result is float.
```

Hierarchy of Types

char $\rightarrow$ int $\rightarrow$ float $\rightarrow$ double

Conversion happens from lower to higher type.

Type Conversion Error

- loss of precision
- Overflow or Underflow.
- Invalid conversion.

Role in Compiler Design

- Used during semantic analysis.
- Helps in type checking.
- Ensures correct execution of expression.

Advantage of Type Conversion

- Flexibility in Programming
- Supports mixed-type expressions
- Improves usability

OVERLOADING OF FUNCTION & OPERATORS

FUNCTION OVERLOADING

Function overloading is a feature in which multiple functions can have the same name but different parameters.

The compiler identifies the correct function based on:

- Number of arguments.
- Type of arguments.
- Order of arguments.

Purpose of Function Overloading

Improves code readability.

Reduces need for different function names.

Supports polymorphism.

Example -

```
#include <iostream>
using namespace std;

class Demo {
public;
```

```

int add (int a, int b) {
    return a + b;
}

int main() {
    Demo d;
    cout << d.add(2, 3) << endl;
    cout << d.add(2.56, 3.56) << endl;
    return 0;
}

```

How Compiler Handles it

The compiler checks:

- Function signature.
- Parameter list.

Then selects the appropriate function.

Advantages

- 1) Same function name for related operations
- 2) Easier programming
- 3) Better readability

Disadvantages.

- 1) may create ambiguity
- 2) Increases compiler complexity

OPERATOR OVERLOADING

Operator Overloading means giving special meaning to operators for user-defined data types (classes/objects).

Same operator performs different operations depending on operands.

Purpose:-

- makes objects behave like built-in types.
- Improves code clarity.
- Simplifies complex operations.

Common Overloaded Operators $\rightarrow +, -, *, /$
 $=, <, >.$

Advantages

- Improves readability
makes code intuitive
- Supports object-oriented programming.

Disadvantages

- Can make code confusing.
- Overuse reduces clarity.

POLYMORPHIC FUNCTION. [May 2023]

A Polymorphic function is a function that can work with different types of data without changing its name or writing separate functions again & again.

It behaves correctly for many data types like integer, float, character, string etc.

Example-

Suppose we make a function called `add()`.

It can

- add integers
- add floating numbers
- join strings!

`add(2, 3) → 5`

`add(2.5, 3.1) → 5.6`

`add("Hi", "Au") → "Hi Au".`

Same function name → different behaviour.

In Compiler Design-

In Compiler Design, polymorphic functions are important because the compiler must understand:

- which version of function to use.
- what type of data is passed.
- whether operations are valid or not.

Types of Polymorphism-

1. Compile-Time Polymorphism → The compiler decides the function during compilation.

Also called:

- Static Polymorphism
- Early Binding

Example:-

```
int add (int a, int b)
```

```
float add (float a, float b)
```

→ The compiler chooses the correct function based on arguments.

2. Run-Time Polymorphism → The decision happens while the program is running.

Also called:

Dynamic Polymorphism
Late Binding.

Example:

Using virtual function in C++.

RUNTIME ENVIRONMENT

STORAGE ORGANISATION.

Storage Organisation means How a program's data & variables are arranged and managed in computer memory while the program is running.

The compiler decides:

- where variables will be stored.
- How much memory they need.
- and when that memory should be created & removed.

Importance of Storage Organization

A program uses many things in memory:-

- variables
- functions
- arrays
- objects

If memory is not organized properly:-

- program becomes slow
- memory gets wasted
- or program may crash.

So, storage organization helps in :-

- efficient memory use
- fast execution
- proper function calling.

STORAGE ALLOCATION STRATEGIES [June 2025]

Storage allocation strategy means: [May 2024]

The method used by a compiler to allocate memory to variables & functions during program execution. [May 2023]

It tells:

- where memory will be given
- when memory will be given
- & when memory will be removed.

Need of Storage Allocation-

A program contains:

- variables
- functions
- arrays
- objects

All these needs memory.

Compiler uses storage allocation strategies to-

- use memory efficiently.
- run programs correctly.
- manage function calls.

Types of Storage Allocation Strategies

There are mainly 3 types:

1. Static allocation [may 2023]
2. Stack allocation [may 2023]
3. Heap allocation [may 2023]

1. Static Allocation

In Static Allocation memory is allocated before program execution starts.

memory size remains fixed throughout execution.

It is used for-

- global variables
- static variables
- fixed size data.

Example:-

```
int x = 10;
```

```
Static int y = 5;
```

memory for x and y is allocated once & remains till program ends.

Features -

- memory allocated only once.
- Fixed memory size.
- Variables exist throughout program execution
- Simple & fast.

Advantage -

- 1) Fast Allocation $\rightarrow$ memory is already reserved.
- 2) Simple Implementation $\rightarrow$ Easy for compiler to manage.
- 3) No Runtime Overhead $\rightarrow$ No extra work during execution.

Disadvantage -

- 1) memory wastage $\rightarrow$ unused memory cannot be reused.
- 2) No Flexibility $\rightarrow$ Size can't change during run time.
- 3) Recursion Problem $\rightarrow$ Recursive functions cannot be handled properly.

2) Stack Allocation

In Stack Allocation memory is allocated when a function is called & removed automatically when function ends.

It uses a stack data structure.

↳ Stack works on LIFO.

It is used for -

- local variables
- function parameters
- return addresses
- recursive functions

Example :-

```
void fun()  
{  
  int a = 5;  
}
```

→ Variable a is created when function starts & destroyed when function ends.

Working -

```
main()  
└─ fun1();  
    └─ fun1()  
        └─ fun2();
```

Step-1:

main() gets memory in stack.

Step-2:

fun1() is called → another block added.

Step-3:

fun2() is called → another block added.

Step-4:

fun2() ends → its memory removed.

Step-5:

fun1() ends → its memory removed.

Stack Frame / Activation record - (2025)

Each function call creates a block called:
Stack Frame (Activation Record).

It stores:

- local variables
- parameters
- return address
- temporary data.

Features-

- Dynamic allocation during runtime
- Automatic memory management
- Supports recursion
- fast access.

Advantages -

- 1) Efficient memory use $\rightarrow$ memory reused automatically.
- 2) Supports Recursion $\rightarrow$ Each recursive call gets separate memory.
- 3) Fast Allocation & Deallocation $\rightarrow$ Push and Pop operations are fast.

Disadvantages -

- 1) Limited Size $\rightarrow$ Large recursion may cause Stack overflow.
- 2) Temporary Storage Only $\rightarrow$ Data removed after function ends.

3) Heap Allocation

In Heap Allocation memory is allocated dynamically during program execution according to programmers need.

It is used ~~for~~ when -

- Size is unknown beforehand
- large memory is needed
- dynamic data structures are used.

Example:

```
int *p;
```

```
p = (int *) malloc(sizeof(int));
```

→ memory is allocated from heap.

Important Function:

Function	Purpose
malloc()	Allocates memory
calloc()	Allocates multiple memory blocks.
realloc()	Changes memory size.
free()	Releases memory.

Features:

Dynamic memory Allocation

Flexible size

Memory controlled by Programmer.

Large storage area.

Advantage -

- Memory can increase or decrease.
- Useful for dynamic structures
- Not destroyed automatically.

Disadvantage-

- Allocation takes more time.
- If memory is not freed properly.
- Programmer must handle memory carefully.

PARAMETER PASSING

Parameter passing means the method used to send values or data from a calling function to a called function.

When one function calls another function, it may send some data called parameters or arguments.

Example:-

```
#include <stdio.h>
void add(int a, int b)
{
    print ("%d", a+b);
}
int main()
{
    add(5, 3);
    return 0;
}
```

Here,

- 5 and 3 are arguments.
- a and b are parameters.

- Data is passed from `main()` to `add()`

Types of Parameter Passing

1. Call by value — In this method, a copy of the actual value is passed to the function.

Changes inside the function do not affect the original variable.

2. Call by Reference — In this method, the address of the variable is passed.

Changes inside the function affect the original variable.

Examples:- (1)

```
void fun(int x)
{
    x = 20;
}
int main()
{
    int a = 10;
    fun(a);
}
```

a remains 10.

(2)

```
void fun(int *x)
{
    *x = 20;
}
int main()
{
    int a = 10;
    fun(&a);
}
```

→ a becomes 20

DYNAMIC STORAGE ALLOCATION

[June 2021]

[May 2021]

Dynamic Storage Allocation means memory is allocated to variables during program execution (run time) according to need.

The memory is not fixed before the program starts. It is created & removed while the program is running.

In Simple Language

Suppose a program does not know how much memory it will need in advance.

Then it asks memory from the system while running.

This process is called dynamic storage allocation.

Example:-

```
#include <stdio.h>
#include <stdlib.h>

int main ( )
{
```

```

int *p;
p=(int*) malloc(size of (int));
*p=10;
printf ("%d", *p);
free(p);
return 0;
}

```

Explanation:

- malloc() allocates memory during run time
- p stores the address of allocated memory.
- free() releases the memory.

Features-

- Memory is allocated at run time.
- memory size can change when needed.
- Efficient memory usage
- Uses heap memory.

Advantages-

- Better memory utilization
- Useful for large programs
- memory can be allocated as needed.

- Supports dynamic data structures like:
 - linked list
 - Tree
 - Graph

Disadvantage -

- ~~Some~~ Slower than static allocation.
- Memory leak may occur if memory is not freed. [released memory that can be used again]
- more complex memory management.

ERROR DETECTION & RECOVERY [May 2024]

When a programmer writes a program, mistakes may occur.

The compiler must:

1. Detect the errors
2. Recover from the error
3. Continue checking the remaining programs.

This process is called Error Detection and Recovery.

Error Detection & recovery are mainly performed during:

- Lexical Analysis
- Syntax Analysis
- Semantic Analysis.

A good compiler should:

- Detect errors accurately.
- Show meaningful messages
- Recover efficiently.

Error Detection -

Error Detection is the process of finding errors in a program during compilation.

The compiler checks the program & identifies mistakes like:

- Syntax Error, Lexical Errors, Semantic Errors
- missing symbols
- Wrong statements.

Error Recovery-

Error Recovery is the process of handling errors & continuing compilation without stopping the whole program.

The compiler tries to recover from errors so more errors can also be detected.

Need of Error Recovery

- Prevent compiler from stopping immediately.
- Detect multiple errors in one compilation.
- Help programmer correct mistakes easily.

Advantages -

1. Saves debugging time
2. Improve program correctness
3. Detects multiple errors together
4. Makes compiler user - friendly.

Ad-Hoc & Systematic Method

Ad-Hoc Method

Ad-Hoc method is an informal method of solving problems without using fixed rules or proper structure.

Features -

- No fixed procedure
- Quick solution
- Used for small problems.

Disadvantages -

- Difficult to maintain
- Not organized
- Less reliable

Systematic Method

Systematic method is a structured and organized method that follows proper rules & steps.

In compiler design, predefined algorithms & formal techniques are used.

Features -

- Proper planning
- Fixed Rules & procedures.
- More reliable & efficient

Advantages-

- Easy to maintain
- Better accuracy
- Suitable for large systems

Disadvantages-

- more development time, needs planning and design.
- Complex implementation - Requires knowledge of algorithm & theory.